

Linux プログラミング テスト完全対策まとめ

Linux の基本

Linux とは

- UNIX 系 OS の一種
 - サーバ、組み込みシステム、スマートフォンなどで利用
 - この授業では Linux 上でのシステムプログラミングを学習
-

パス

絶対パス

/home/haru/test.c

ルート (/) から書く

相対パス

../test.c

現在位置から書く

特殊記号

記号 意味

・ 現在のディレクトリ

.. 親ディレクトリ

～ ホームディレクトリ

よく使うコマンド

pwd

現在位置表示

pwd

ls

一覧表示

ls

ls -l

ls -a

-l: 詳細表示

-a: 隠しファイルも表示

cd

移動

cd test

cd ..

cd ~

cp

コピー

cp a.txt b.txt

mv

移動・名前変更

mv a.txt b.txt

rm

削除

rm a.txt

rm -r dir

mkdir

ディレクトリ作成

mkdir test

cat

内容表示

cat file.txt

grep

文字列検索

grep cat file.txt

find

検索

find . -name test.c -print

リダイレクト

標準入力

ファイル記述子 0

標準出力

ファイル記述子 1

標準エラー出力

ファイル記述子 2

出力保存

ls > out

標準出力を保存

エラー保存

```
ls nofile 2> err
```

標準エラー出力を保存

全部保存

```
ls out out2 out3 > allout 2>&1
```

標準出力とエラー出力両方保存

パイプ

前の出力を次の入力へ

```
ls /usr/bin | grep cat
```

実行順

```
ls
```

↓

grep

shell スクリプト

基本形

`#!/bin/bash`

実行権限

`chmod u+x file`

u : user

x : execute

引数

\$1

\$2

\$3

例

`#!/bin/bash`

```
find . -name "$1" -print
```

実行

```
./f test.c
```

man コマンド

マニュアル表示

```
man ls
```

終了

```
q
```

ファイル記述子

OS がファイルを管理するための番号

番号 意味

0 標準入力

1 標準出力

番号 意味

2 標準エラー出力

open

ファイルを開く

```
fd=open("test.txt",O_RDONLY);
```

戻り値

- 成功 → fd
 - 失敗 → -1
-

read

読み込み

```
read(fd,buff,10);
```

write

書き込み

```
write(1,buff,10);
```

close

閉じる

```
close(fd);
```

perror

エラー内容表示

```
perror("open");
```

例

```
open: No such file or directory
```

システムコールとライブラリコール

システムコール

OS 機能

open

read

write

close

fork

ライブラリコール

C ライブラリ

printf

scanf

fprintf

printf 内部では write が使われる

プロセス

実行中プログラム

例

Chrome

VSCode

Terminal

全部別プロセス

fork

プロセス生成

`pid=fork();`

戻り値

親

`pid > 0`

子の PID

子

`pid == 0`

エラー

`pid < 0`

fork 後

親

parent

子

child

同時実行

プロセス番号

確認

ps u

取得

getpid()

シグナル

ソフトウェア割込み

ハンドラ

```
void handler(int signum)
```

```
{
```

```
}
```

登録

```
signal(SIGUSR1,handler);
```

送信

```
kill(pid,SIGUSR1);
```

よく使うシグナル

SIGUSR1

SIGUSR2

スレッド

プロセス内の実行単位

プロセスとの違い

プロセス

メモリ共有しない

スレッド

メモリ共有する

作成

`pthread_create()`

終了待ち

`pthread_join()`

コンパイル

`cc -pthread thread.c`

Mutex

相互排他

lock

`pthread_mutex_lock()`

unlock

`pthread_mutex_unlock()`

クリティカルセクション

共有変数操作部分

例

`count++;`

デッドロック

状況

A

mutex1 保持

mutex2 待ち

B

mutex2 保持

mutex1 待ち

結果

永久停止

発生条件

1. 相互排他
2. 保持待ち
3. 横取り不可
4. 循環待ち

ライブロック

停止しない

しかし

仕事が進まない

例

lock 失敗

↓

unlock

↓

再挑戦

↓

lock 失敗

永久ループ

スターベーション

飢餓状態

例

優先度高いスレッドだけ実行

低優先度スレッドが永遠に動けない

Git

ワーキングディレクトリ

編集場所

↓

インデックス

git add

↓

ローカルリポジトリ

git commit

↓

リモートリポジトリ

git push

よく使うコマンド

git init

git add

git commit

git push

git status

git log

git diff

reset

履歴を戻す

git reset --hard HEAD^

revert

打消しコミット作成

git revert HEAD

超重要予想問題

Q1. ファイル記述子 0,1,2 を説明せよ

A.

0:標準入力

1:標準出力

2:標準エラー出力

Q2. fork の戻り値は？

A.

親→子 PID

子→0

失敗→-1

Q3. signal と kill の役割

A.

signal：受信設定

kill：送信

Q4. pthread_create の役割

A.

スレッド生成

Q5. mutex の目的

A.

共有資源への同時アクセス防止

Q6. デッドロックとライブロックの違い

A.

デッドロック：

全員停止

ライブロック：

動いているが進まない

最終暗記

0 → 標準入力

1 → 標準出力

2 → 標準エラー出力

fork

親→PID

子→0

signal

受信

kill

送信

pthread_create

生成

pthread_join

待機

mutex

lock

unlock

デッドロック

停止

ライブロック

進まない